Kipu Anon · Doc de diseño (Reto 1: Vende los datos sin vender a la gente)

Ismael Florean · Frontier CDMX/26 · Repo: github.com/SamaelFL/kipu-anon · Diagrama: docs/diagrama.png

1. Problema, meta y tesis

Hay que entregar el corpus de Kipu Pay (2.028 mensajes, 26 docs, 6.243 filas) uno a uno, sin que nadie sea reidentificable, útil para entrenar y con cada persona como una sola identidad de punta a punta, incluso cuando las fuentes crecen. Puntúa fuga 45, utilidad 35 y consistencia 20; una base solo con reglas da ~88. **Tesis:** el problema no es encontrar nombres, es **saber quién es quién**. Lo revisamos sobre el corpus real: hay 8 nombres repetidos en `users.json`, una agente que se llama igual que un cliente, apodos ("Pipe", "Teban") y apellidos que también son lugares ("Santander", "Castilla", "Restrepo", "Bolívar" aparecen como barrios en transacciones). Por eso el centro del diseño es un **grafo de identificadores**: los nombres son etiquetas, nunca llaves. Encima van reglas deterministas, un LLM de recall cacheado y un juez local que replica al del reto. El texto se reemplaza, nunca se reescribe.

2. Pipeline (Python con librería estándar; mismo código para `/run` y `/anonymize`)

1. **Ingesta exhaustiva:** API con ETag, backoff y caché. Se leen todos los rincones donde vimos datos personales en el corpus: Info de PDF en UTF-16 (Author, Subject, **Keywords** con nombres), comentarios HTML ("revisado por … pagudelo@gmail.com") y comentarios XML, front matter con `@handles` y correos de terceros, `<meta author>`, nombres de adjuntos, y URLs con slugs o parámetros (`?u=khenao`, `/comercios/gabriela-londono`). `clientes.csv` va en Latin-1. Salida 1:1 con `record_index`.
2. **Grafo de identificadores:** los nodos son llaves fuertes (CL-, KP-, uid, radicado PQR, cédula/CE, correo, celular, placa, pasaporte, n.º de crédito `4471-xxxx`) y las aristas salen de las tablas, de `users.json` (correo → cliente) y de las transacciones (cliente → crédito). Cada componente es una persona con su género y todas sus formas: completo, nombre + apellido, pila, MAYÚSCULAS sin tildes, apodo de `display_name`, handle, slug, local del correo, cédula con y sin puntos, celular con y sin 57, dirección, nacimiento.
3. **Detección en capas:** (a) diccionario de formas con la normalización exacta del juez (NFKC, confusables, invisibles, tildes, separadores de dígitos); (b) patrones: correo, correo deletreado ("martin punto torres arroba icloud punto com"), cédula, celular, placa (`KFY046`, `JXA-971`), pasaporte, crédito, dirección colombiana (Calle, Cra, Tv, Diagonal # n-n), honorífico + apellido ("Estimado señor Santander"); (c) coincidencia tolerante (edición 1) para copias alteradas; (d) **LLM por hilo o documento**, en paralelo y con JSON de spans, cacheado por hash: recall de terceros fuera de las tablas, desempate persona vs. lugar y oficio → sector.
4. **Resolución con evidencia**, nunca por nombre: llave en el registro (uid del autor, CL-, radicado) > identificador fuerte en el registro o el hilo > **fecha + monto que cruza con transacciones** > adjunto (`extracto_<nombre>_mes`) > participantes del hilo. Un nombre sin evidencia es una persona aparte y se marca para revisión. **Protección de falsos positivos:** un gazetteer construido del propio corpus (ciudades, barrios del maestro, comercios y lugares de transacciones) más una lista de organizaciones (Bancolombia, Nequi, Avianca, Superfinanciera, Kipu Pay con su NIT y su oficina). Un token que es apellido y lugar solo se reemplaza dentro de un nombre o junto a un honorífico.
5. **Seudónimos deterministas:** `HMAC(llave fija, llave de la persona)` → catálogo propio colombiano del mismo género; el género se desempata con el cargo u oficio ("Contadora"). Cada forma corta se mapea a la parte equivalente del seudónimo ("Manuela" → "Sofía Elena"), el apodo va al nombre de pila del seudónimo, y el handle, slug y correo conservan el patrón (`nombre.apellido`, inicial + apellido, guiones). Cédula, celular, placa, pasaporte y crédito con formato válido y lejos del original. Nunca coincide con una persona real. Las colisiones se resuelven sin depender del orden y el mapa se persiste, así que nadie cambia de seudónimo cuando crecen las fuentes.
6. **Reemplazo con formato:** spans de mayor a menor; se respetan puntos, mayúsculas, snake_case, la forma deletreada ("arroba/punto") y la URL. Se reemplaza también en claves de metadata, adjuntos y comentarios. El `reporta_a` de empleados usa la misma forma corta del seudónimo del jefe. Ningún registro se vacía.
7. **Cuasi-identificadores**, calibrados contra la salida ideal. En el maestro, siempre: oficio → sector, barrio → otro barrio de la misma ciudad, dirección y nacimiento → seudónimos; la ciudad, la vinculación y las llaves se quedan. En el texto: el oficio, barrio y año de vinculación de la persona → su sector, su ciudad y "hace unos años". Además se detectan **tuplas sin nombre** ("cliente de Bucaramanga, arquitecto, vinculado desde 2021") y, si apuntan a menos de k clientes del maestro, se generalizan igual.
8. **Verificación adversarial antes de publicar:** (a) barrido global de todas las formas reales con la normalización del juez; (b) ataque de enlace: se intenta reidentificar a cada cliente cruzando los cuasi-identificadores de la salida contra el maestro original; (c) `validar.mjs`; (d) `entities.json` honesto: todas las formas, registros y los conservados con `kept` y `reason`. Si algo falla, se corrige y se vuelve a verificar.

3. Servicio

Rutas en `/v1` con `ThreadingHTTPServer`; Bearer opcional. `/run` corre en un hilo y reporta estado, conteos y costo; `/output` y `/entities` salen del volumen. `/anonymize` carga en memoria el grafo, el mapa y la caché: la misma persona, aunque venga alterada, recibe el mismo seudónimo; una nueva recibe uno nuevo y único. Corte de 40 s: si el LLM no responde, sigue con reglas. Railway con 1 réplica, sin reposo, volumen y healthcheck.

4. Decisiones y trade-offs

Elegimos	Descartamos y por qué
Grafo de identificadores; el nombre es solo etiqueta	Agrupar por nombre: fusiona homónimos y a la agente con el cliente
Reglas + LLM de recall cacheado	Solo reglas: ~88. LLM end-to-end: no determinista y reescribe
Gazetteer sacado del corpus	Lista genérica: no conoce los barrios ni los comercios de Kipu
Generalizar siempre, ligado a la persona	k-anonimato condicional: se cruza entre fuentes y se aleja de la ideal
HMAC + mapa persistido	Seudónimos al azar: rompen consistencia entre corridas
Juez local + ataque de enlace	Iterar a ciegas con 3 corridas

5. Seguridad y fallas

- El texto de las fuentes es dato: el LLM solo devuelve spans con un esquema fijo; nunca recibe instrucciones de las fuentes ni ejecuta nada.
- Llaves solo en variables de entorno; sin hash sin sal; `entities.json` nunca viaja con el dataset.
- LLM caído → reglas. 429 → backoff. Reinicio → el estado sigue en el volumen. Esquema inválido → 422. Handler global.
- **Fuentes que crecen:** el canario de `record_index` lo detecta. El grafo se extiende sin renumerar, y si una persona vieja aparece con un identificador nuevo, se une a su componente. La corrida 1 va antes del cambio (30 %).

6. Costo y evaluación

El LLM ve ~250 mil tokens (chats, docs, notas, descripciones): unos US$1 y menos de 15 min con 8 hilos; después, la caché lo deja casi en cero. **Juez local:** copia la normalización de `validar.mjs` y mide sobre `sample/` contra la salida ideal (partición de personas, fugas, parecido, conservados), los 13 tests de la muestra, los 5 públicos y una batería de trampas sacada del corpus real: homónimos, agente = cliente, apodos, apellido-lugar, deletreados, placas, URLs, metadata UTF-16 y tuplas sin nombre. Todo se reporta por familia.

7. Límites

Un nombre de pila suelto sin evidencia queda como persona aparte: preferimos separar a fusionar. Si un PDF trae fuentes embebidas raras, el parser stdlib necesitaría un extractor de respaldo.